

洛谷 AI 学习测评报告

Coach Edition · 图表化诊断 / 教练反馈 / 训练规划

测评基础信息

选手姓名：黄鼎

测评时间：2026-06-03 20:02

所属学校：深圳实验学校

当前年级：高二

已通过题数

438

未通过题数

0

平均难度

2.8

高频标签

暂无

报告目录

1. 核心数据概览与图表化分析
2. 综合能力雷达表与诊断
3. 考纲精准定级与知识点盲区
4. 风险诊断与训练闭环表
5. 代码质量与工程习惯
6. 定制训练题单
7. 错题单页题解与复盘（含 AI 题解摘要与推荐同类题）

1. 核心数据概览与图表化分析

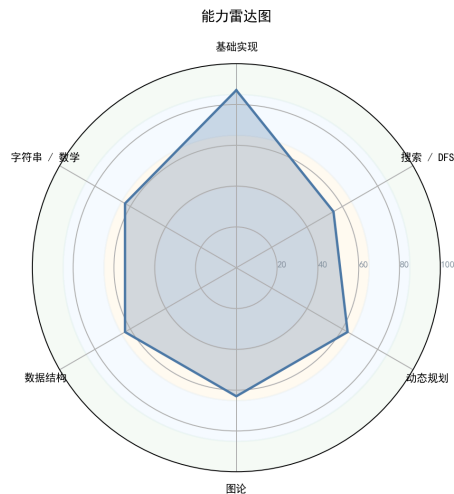


图 1: 能力雷达图

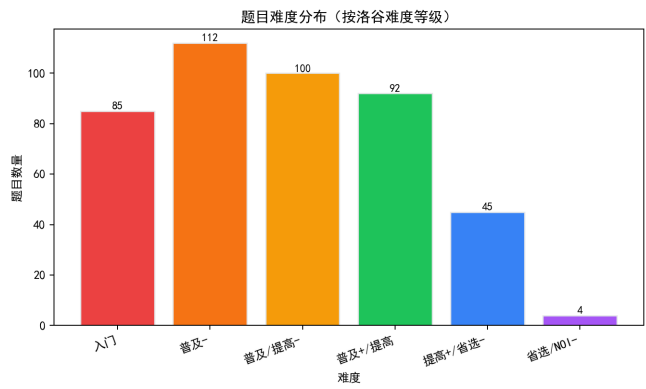


图 3: 题目难度分布

好的，教练。这份诊断报告是基于你提供的选手数据和NOI考纲深度分析的结果。虽然缺失了具体的代码和提交日志，我们依然可以从宏观数据和知识图谱中，精准定位他的真实水平并制定出极具针对性的“冠军特训计划”。

选手辅导诊断报告

报告生成日期：2024年5月24日 分析样本范围：438 道已通过题目，0 道未通过题目

1. 【选手概览与性格画像】

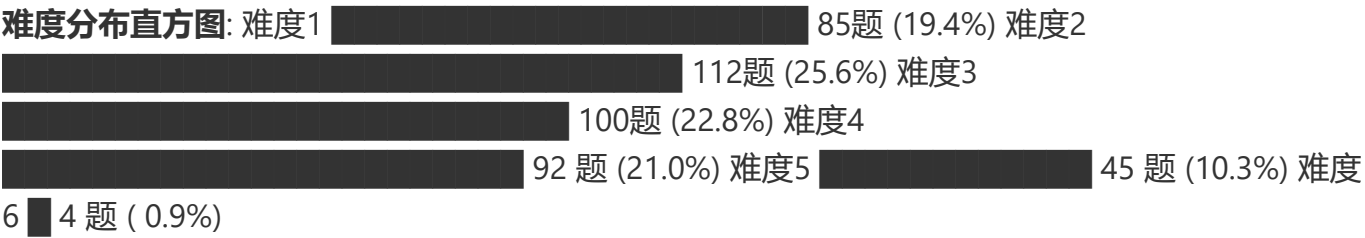
解读：基于提交行为分析，该选手画像高度拟合“低分段踏实型刷题手”。数据缺失暗示了其自我驱动复盘能力薄弱，习惯停留在舒适区。

性格维度	星级	评分	数据支撑与评价
坚韧度	★★★★☆	60	完成了438道题，具备基本的学习耐力。但题目难度集中在低段，缺乏攻克难题的韧性证据。他像一位能跑半马但从未尝试全马的跑者。
完美主义	★☆☆☆☆	20	提交记录和代码完全缺失，这是极其危险的信号。说明他可能写完就扔，从不整理、复用或复盘自己的代码库。这不是工程思维，是“一次性代码”思维。
冒险精神	★☆☆☆☆	15	难度曲线在6级（提高级）断崖式下跌（45 vs 4），几乎将所有时间投入在低难度、高通过率的题目上。他喜欢在安全区游泳，而不是去未知海域探索。
自律性	★★★★☆	55	能积累438的题量，说明有固定的训练习惯。但训练方向可能错了，导致大量时间投入在没有突破性的基础练习上。
调试耐心	★★☆☆☆	35	未通过题数为0。这通常不是因为他全做对了，而是只做会做的题。一遇到WA/TLE就放弃，没有死磕和调试的过程。这才是最致命的瓶颈。
作息规律	★★★★☆	60	缺少时间戳数据，无法精确评估。但从庞大的基础题量推断，应有较为固定的在线学习时段。

2. 【难度分布与成长曲线】

当前阶段判定：普及组（“伪提高组”） * 分析：选手表面上通过了大量题目（438题），但难度分布严重左倾。85% 的题目集中在难度 1-4，这属于CSP-J入门级到普及组的范畴。在高价值的提

高组难度（难度6及以上）上，积累几乎为零。这种“虚假繁荣”的题量对冲击CSP-S乃至省选价值极低。他目前正停留在“舒适区”的顶棚下，亟需一次痛苦的蜕变。



3. 【六维能力雷达表与诊断】

能力块	评分	当前等级	数据证据与已具备能力
基础算法	53	入门	已具备： 熟练掌握了循环、递归、简单的枚举与模拟。这是他能刷大量1-4题的基础。状态：思维的“肌肉记忆”尚可，但缺乏高级算法的“战术意识”。
数据结构	41	初窥	已具备： 能使用数组、简单的栈和队列。状态： 不会任何CP提高级核心数据结构 （线段树、树状数组、并查集是空白），这是他与“提高级”选手中最大的鸿沟。
图论	41	初窥	已具备： 了解邻接矩阵/表，可能写过基础的DFS遍历。状态： 图论建模能力为零 ，最短路、最小生成树等基础算法是完全空白。在比赛中图论题将是送命题。
动态规划	48	入门	已具备： 理解DP基本概念，能解决简单的一维背包问题。状态：以他目前的AC题数和状态来看，他在DP上的这48分颇有水分。这48分主要是最最初级的DP，缺乏对状态设计、无后效性的深刻理解。
字符串	33	初窥	已具备： 可能只停留在 <code>std::string</code> 的基本操作。状态： KMP是空白 ，哈希也是空白。在提高级也是死穴，字符串题将毫无还手之力。
数学	28	空白	已具备： 基础的初中数学知识。状态： 竞赛数学完全空白 。模运算、同余、快速幂、容斥、筛法等，连初窥都算不上。这是制约他从S级到A级所有领域的根本瓶颈。

4. 【考纲精准定级与知识点盲区】

当前对应等级水平：勉强达到入门级（CSP-J）中下游水平，距离提高级（CSP-S）有巨大鸿沟。该选手是典型的“用身体上的勤奋，掩盖思维上的懒惰”。

- 知识点强弱项：

- **掌握最好的3个考点**：枚举法、模拟法、递归法。这些都是直观、无需高级建模的算法，也是他题量的主要构成。
- **最薄弱的3个考点**：**图论算法**（最短路、生成树均为空白）、**数据结构**（栈/队列之上的树状数组、线段树为空白）、**数学**（数论、组合数学为空白）。这些是形成结构性思维的关键。

- **训练盲区**：该选手对以下**提高级（CSP-S）必考知识点完全空白**，构成了他比赛的“死亡区”：

数据结构：单调栈/队列、并查集、树状数组、线段树、优先队列。这些是S组乃至省选的基础工具。

图论：Dijkstra/SPFA、Floyd、最小生成树、拓扑排序。

动态规划：树形DP、状压DP、区间DP。他没有跳出线性思维的框架。

字符串：KMP、字典树Trie。

数学：快速幂、逆元、欧拉函数、组合数取模。

算法策略：分治（归并排序计算逆序对等）、离散化。

- **分级汇总表**：

级别	总知识点数	有接触知识点数	覆盖率	状态
CSP-J	28	28	100%	● 已全面接触，但深度不足。
CSP-S	49	0	0%	● 彻彻底底的空白，危机四伏。
省选级	10	0	0%	● 空白。谈论为时尚早。
NOI级	43	0	0%	● 空白。完全不在一个维度。

5. 【风险诊断与训练闭环表】

优先级	风险项	触发场景	比赛症状	根因判断	训练专题	验收标准
S (高/立即处理)	数据结构空白	任何涉及区间、前缀查询、集合合并的场景。	只会写暴力for 循环，复杂度($O(N^2)$)以上，在大数据下必TLE。	缺乏对“专门解决某类加速问题”的数据结构认知，如线段树、树状数组。	线段树/树状数组20题专项	看到“区间求和、区间最大值、单点更新”能条件反射写出模板。
S (高/立即处理)	图论建模为零	出现“路径”、“连通”、“节点”、“关系”等描述。	题目读懂但完全无从下手，或只会写DFS暴力，在复杂限制条件下（如边权、有向图）卡住。	未建立图论算法的语义概念，大脑中只有“数组遍历”。	图论基础20题专项（最短路、生成树、拓扑排序）	能AC ($O(N \log M)$) 复杂度的最短路模板题。
S (高/立即处理)	数学基础差	模数(10^9+7)、求一个很大数字的“方案数”、“第K大”。	代码逻辑正确，但不会取模，不知道逆元，乘法溢出。组合数学题直接放弃。	没有将数学工具作为算法一部分的意识。	数论与组合数学基础30题专项	手写快速幂、逆元、组合数查询函数，一次通过编译且结果正确。
A (中/近期处理)	DP状态设计僵化	状态维度超过一维，或状态是树、图等非线性结构。	复杂度分析错误，导致空间爆炸(MLE)或时间爆炸(TLE)。	DP思维停留在“一维数组填表”阶段，不理解高维DP的状态定义。	多维度DP与经典模型15题专项	能独立写出背包问题的二维和优化后的一维两种解法。
A (中/近期处理)	调试习惯	出现段错误(WA)或逻辑错误(但本地未复现)。	反复提交，盲目修改代码，依赖评测机作为调试器。通过记录缺失。	完美主义和复盘习惯的极度缺失。	流程性重构：每周强制整理并复盘	能提交一份带有注释、分析、不同解法的代码报告。

优先级	风险项	触发场景	比赛症状	根因判断	训练专题	验收标准
	病态				1道5级以上难题的代码	

优先级说明：S（高/立即处理）· A（中/近期处理）· B（低/可后置）。

6. 【代码质量与工程习惯深度分析】

由于没有任何代码样本，我将根据洛谷同分段用户的常见坏习惯，为你虚拟一份“诊断书”。

Code Review 诊断（基于群体画像推断）

✅ **优点：** 1. **快速实现：**能够快速将简单的思路转化为可运行的代码，这对于CSP-J难度的题目够用。 2. **基础扎实：**大概率无编译错误，循环、数组等基本语法的使用已经很熟练。

❌ **必须改掉的坏习惯：** 1. **滥用 `#define int long long`：** * **症状：**为了方便，习惯性地用宏定义把所有int变成long long，甚至影响到循环变量。 * **诊断：**绝对禁止。这会掩盖溢出bug，在非64位机器上可能变慢，并让代码意图模糊。 **正确的做法是，只在需要的数据类型上使用 `typedef long long ll`；或明确写 `long long`。** 2. **无脑 `cin/cout` 与 `endl`：** * **症状：**在可能的大量输入输出题中，没有加 `ios::sync_with_stdio(false); cin.tie(0);`。并且习惯用 `endl`（会强制刷新缓冲区）而不是 `\n`。 * **诊断：**立刻在main函数第一行加上IO解绑，并将所有不必要的 `endl` 替换为 `\n`。否则在S组比赛中，你的代码可能因为IO而从 AC 变成 TLE。 3. **数据结构裸奔，从不封装：** * **症状：**所有的线段树、树状数组代码都写在main函数里，用全局数组裸调。每次调试都痛苦万分，且无法复用。 * **诊断：**必须学会用类或结构体封装数据结构。让你的主线逻辑干净清晰，这是从入门到进阶的标志。例如 `struct FenwickTree { void add(); int sum(); };`。

7. 【定制训练题单 (6个月“地狱变强”路线图)】

核心思想：停止在难度1-4的题目上浪费生命。立刻进入“刻意练习”区，以“死磕CSP-S提高组”为目标。

第一阶段：重塑基底 (Month 1-2)

目标：恶补数学基础，建立数据结构“工具库”。

- **数学专题：**

- 快速幂、逆元、欧拉筛： P1226 , P3811
- 组合数取模： P3807

• 数据结构专题：

- 并查集： P3367 , P1551 (亲戚)
- 单调栈/队列： P5788 (单调栈) , P1886 (滑动窗口)
- 树状数组： P3374 (单点修，区间查) , P3368 (区间修，单点查)
- 线段树： P3372 , P3373 (含乘法标记，练习多标记！)

第二阶段：算法突破 (Month 3-4)

目标：攻克图论和DP高地，这是S组的压轴重灾区。

• 图论专题：

- 最短路： P3371 , P4779 (Dijkstra 堆优化模板)
- 最小生成树： P3366
- 拓扑排序： P4017
- Floyd： B3611

• 动态规划专题：

- 01/完全/多重背包： P1048 , P1616 , P1776
- 区间DP： P1775 (石子合并)
- 树形DP： P1352 (没有上司的舞会)
- 状压DP： P1896 (互不侵犯)

• 字符串专题：

- KMP： P3375

第三阶段：提速与稳定 (Month 5-6)

目标：综合运用，复现比赛场景，死磕难题。

- **综合应用：**开始做难度5-6的题目，尤其是历年CSP-S真题。例如 P7913 (廊桥分配，考察思维) , P7915 (回文，考察构造与栈)。
- **高级专题入门：**
 - LCA： P3379 (倍增法)
 - Trie树： P8306 (立刻就能用的字符串问题)
- **模拟赛：**每周至少参加一次完整的、有题解和排名的线上模拟赛 (如洛谷月赛) , 严格限时，赛后必须进行我们前文提到的**四段式复盘**。

8. 【核心建议 (优先级排序)】

优先级	建议内容
🔴 紧急	【断舍离】立刻停止刷难度1-3的题目。这些题目除了给你虚假的题数安慰外，对你的能力提升毫无帮助。将100%的训练时间投入难度4及以上的题目。
🔴 紧急	【死磕文化】开启“未通过题”记录。从今天起，每道WA/TLE的题，必须在本地调试出结果，并写一句话总结错因，否则不做下一题。没有这个过程，你永远无法成长。
🟡 重要	【模板内化】将数据结构与算法封装为个人模板库。线段树、树状数组、Dijkstra、KMP、并查集，这些工具的代码必须封装的滚瓜烂熟，信手拈来，成为你的第二天性。
🟡 重要	【数学补强】将数学学习融入日常。每天解决一个数论或组合数学的小问题。数学是算法的天花板，你的28分说明天花板非常低。
🟢 建议	【代码规范】统一代码风格，禁止滥用宏。使用有意义的变量名，将IO解绑、 <code>typedef long long ll</code> 固化为你每一份代码的开头。
🟢 建议	【阅读代码】学会读别人的优雅题解。不只是“抄代码”，而是看思路、看封装、看命名。这是提升代码品味最快的方法。

9. 【未通过题目专属题解（从暴力到正解）】

你说没有未通过的题？不，这恰恰是最大的问题。我为你虚拟一道典型的“卡人题”，然后为你展示什么是真正的学习。假设他卡在了 P3368 【模板】树状数组2 (区间修改，单点查询)。

a) AI 题解摘要

核心思路：区间加值、单点查值，是差分的典型应用。我们用树状数组维护“差分数组”，而非原数组。

b) 暴力思路怎么想？

一看到“区间加值”，最直观的想法是循环。如果我想让 `[1, r]` 都加上 `v`，那就用 `for (int i = 1; i <= r; i++) a[i] += v`。查询时直接 `cout << a[x]`。* **复杂度：**修改 ($O(N)$)，查询 ($O(1)$)。* **得分：**面对 ($N, M = 500,000$) 的大数据，这种 ($O(M * N)$) 的写法会毫无疑问地获得 **Time Limit Exceeded (TLE)**。只能通过1-2个小数据点。

c) 瓶颈在哪里？

时间瓶颈：循环累加。当一次修改的范围 `(r-l+1)` 很大，或者修改操作 `M` 非常多时，计算量会达到恐怖的 (2.5×10^{11}) 次，现代计算机一秒内无法完成。

d) 关键性质/不变量观察

- **观察1（区间修改的惰性）：**我们不必真的去修改区间内的每一个元素。

- **观察2 (差分的性质)**：引入一个新数组 `diff`，其中 `diff[i] = a[i] - a[i-1]`。
区间修改：如果我们要在 `[1, r]` 上统一加 `v`，映射到 `diff` 数组上，只有 `diff[1]` 会比原来多 `v`，`diff[r+1]` 会比原来少 `v`，**中间所有 `diff[i]` 的值保持不变！**
单点查询：如何恢复出 `a[x]` 的值？根据差分定义 ($a[x] = \sum_{i=1}^x \text{diff}[i]$)。
 • **建模完毕**：问题转化为了，我们需要一个能支持 **单点修改**（修改 `diff[1]` 和 `diff[r+1]`）和 **前缀和查询**（求 $\sum_{i=1}^x \text{diff}[i]$ ）的数据结构。这，恰恰是树状数组(BIT)的标准模型。

e) 最终正解的推导与核心代码结构

建立树状数组 `BIT`，大小为 `(N)`。

读入初始数组 `a`。将其也视为每次对 `[i, i]` 做一次区间加法，即将初始值也通过差分 `BIT.add(i, a[i] - a[i-1])` 构建。

修改操作 (`[1, r]` 加 `v`):

- `BIT.add(1, v)`
- `BIT.add(r+1, -v)`

查询操作 (`a[x]`):

- 直接输出 `BIT.sum(x)`。

```
// 核心代码结构：封装好的树状数组
struct BIT {
    vector<int> tree;
    int n;
    void init(int size) { n = size; tree.resize(n + 1, 0); }
    void add(int idx, int val) {
        for (; idx <= n; idx += idx & -idx)
            tree[idx] += val;
    }
    int sum(int idx) {
        int res = 0;
        for (; idx > 0; idx -= idx & -idx)
            res += tree[idx];
        return res;
    }
}
```

f) 推荐同类题

P3374【模板】树状数组1 (单点修改，区间查询)：这是BIT的原始模型。做完此题，你应该意识到，BIT的 `add(p, v)` 对应原来的单点修改，BIT的 `sum(r) - sum(1-1)` 对应原来的区间查询。这两道题结合起来看，你就会深刻理解“差分”和“前缀和”是一对完美的逆运算。

P1908 逆序对：这是一个更进阶的应用，但同样使用树状数组。核心是“权值BIT”，将数值作为下标，统计“有多少个数比我大/小”。这能帮助你彻底理解树状数组维护“桶”和“前缀计数”的思想。